

Assignment-13

Task-1:

Prompt:

Refactor the following legacy code to use a more Pythonic approach, such as list comprehensions or helper functions. Ensure the output remains the same.

```
numbers = [1, 2, 3, 4, 5]  
squares = []  
for n in numbers:  
    squares.append(n ** 2)  
print(squares)
```

Expected Output:

```
[1, 4, 9, 16, 25]
```

Code and Output:

```
Assignment-13.py > ...
1  # Original list of numbers
2  numbers = [1, 2, 3, 4, 5]
3
4  # Using a list comprehension to compute squares:
5  # This iterates over each number 'n' in 'numbers' and computes 'n ** 2'.
6  squares = [n ** 2 for n in numbers]
7
8  # Print the result which will be [1, 4, 9, 16, 25]
9  print(squares)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

[1, 4, 9, 16, 25]
PS D:\AI Assisted Coding> |

Explanation:

- 1. Initial List of Numbers:**
The code starts with a list `numbers` containing the values `[1, 2, 3, 4, 5]`.
- 2. Original Approach:**
The legacy code uses a for-loop to iterate over each element in `numbers` and appends the square of the element (`n ** 2`) to an initially empty list called `squares`.
- 3. List Comprehension:**
The refactored version replaces the for-loop with a list comprehension:
 - The expression `[n ** 2 for n in numbers]` achieves the same result in a single line.
 - It iterates over each `n` in `numbers`, computes `n ** 2`, and directly builds a new list with the resulting squares.
- 4. Printing the Result:**
Finally, `print(squares)` outputs the list `[1, 4, 9, 16, 25]`, which matches the expected output.

Task 2:

Prompt:

Refactor the following legacy code to simplify string concatenation using `" ".join()` for better efficiency and readability. Keep the output unchanged

```
words = ["AI", "helps", "in", "refactoring",  
"code"]
```

```
sentence = ""  
  
for word in words:  
  
    sentence += word + " "  
  
print(sentence.strip())
```

ExpectedOutput:

AI helps in refactoring code

Code and Output:



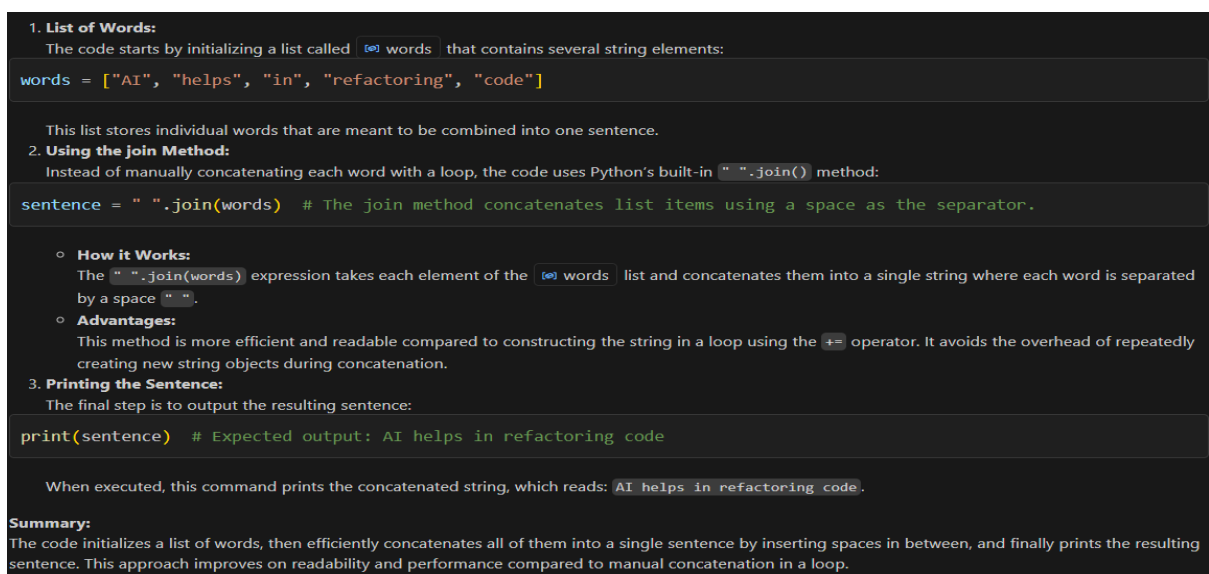
The screenshot shows a Python IDE with a dark theme. The editor window displays a file named 'Assignment-13.py' with the following code:

```
12 # Original list of words  
13 words = ["AI", "helps", "in", "refactoring", "code"]  
14  
15 # Use join to combine the list elements into a single string with spaces between words.  
16 sentence = " ".join(words) # The join method concatenates list items using a space as the separator.  
17  
18 print(sentence) # Expected output: AI helps in refactoring code
```

The bottom panel of the IDE shows the 'TERMINAL' tab with the following output:

```
PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/Assignment-13.py"  
AI helps in refactoring code  
PS D:\AI Assisted Coding>
```

Explanation:



1. List of Words:
The code starts by initializing a list called `words` that contains several string elements:

```
words = ["AI", "helps", "in", "refactoring", "code"]
```

This list stores individual words that are meant to be combined into one sentence.

2. Using the join Method:
Instead of manually concatenating each word with a loop, the code uses Python's built-in `" ".join()` method:

```
sentence = " ".join(words) # The join method concatenates list items using a space as the separator.
```

- How it Works:**
The `" ".join(words)` expression takes each element of the `words` list and concatenates them into a single string where each word is separated by a space `" "`.
- Advantages:**
This method is more efficient and readable compared to constructing the string in a loop using the `+=` operator. It avoids the overhead of repeatedly creating new string objects during concatenation.

3. Printing the Sentence:
The final step is to output the resulting sentence:

```
print(sentence) # Expected output: AI helps in refactoring code
```

When executed, this command prints the concatenated string, which reads: `AI helps in refactoring code`.

Summary:
The code initializes a list of words, then efficiently concatenates all of them into a single sentence by inserting spaces in between, and finally prints the resulting sentence. This approach improves on readability and performance compared to manual concatenation in a loop.

Task 3:

Prompt:

Refactor the following legacy code to use a safer and more Pythonic dictionary access method like `.get()` to handle missing keys gracefully. Ensure the output remains the same.

```
student_scores = {"Alice": 85, "Bob": 90}  
if "Charlie" in student_scores:  
    print(student_scores["Charlie"])  
else:  
    print("Not Found")
```

Expected Output:

Not Found

Code and Output:

```
Assignment-13.py > ...  
1 # Define the student scores dictionary with two entries.  
2 student_scores = {"Alice": 85, "Bob": 90}  
3  
4 # Use .get() to try to fetch Charlie's score.  
5 # The first argument is the key ("Charlie") we want to lookup.  
6 # The second argument is the default value ("Not Found") returned if the key doesn't exist.  
7 score = student_scores.get("Charlie", "Not Found")  
8  
9 # Print the retrieved score or the default message.  
10 print(score)
```

Explanation:

The code in this file performs the following steps:

1. Define the Dictionary

The code defines a dictionary named `student_scores` with two key-value pairs:

- o "Alice": 85
- o "Bob": 90

Each key represents a student's name, and its corresponding value represents their score.

2. Retrieve a Value Safely Using `.get()`

The code uses the dictionary method `.get()` to attempt retrieving the score for "Charlie":

- The first argument is the key to look for ("Charlie").
- The second argument ("Not Found") is the default message returned if "Charlie" is not in the dictionary.

This approach avoids an error that would occur if you tried to access a key with square brackets when the key does not exist.

3. Print the Result

Finally, the code prints the returned value:

- Since "Charlie" is not present in `student_scores`, the `.get()` method returns "Not Found".
- Therefore, the message "Not Found" is printed to the console.

In summary, this code safely tries to access a non-existent key in a dictionary without raising an exception by using the `.get()` method with a default value.

Task 4:

Prompt:

Refactor the following legacy code to replace repetitive if-elif blocks with a dictionary-based approach for cleaner and more scalable logic. Ensure the output remains the same.

```
operation = "multiply"  
a, b = 5, 3  
if operation == "add":  
    result = a + b  
elif operation == "subtract":  
    result = a - b  
elif operation == "multiply":  
    result = a * b  
else:  
    result = None  
print(result)
```

Expected Output:

15

Code & Output:

```
➤ Assignment-13.py > ...  
1 # Define the operation and operands  
2 operation = "multiply" # Current operation  
3 a, b = 5, 3           # Operands  
4  
5 # Create a dictionary mapping each operation to a lambda function performing it  
6 operations = {  
7     "add": lambda x, y: x + y,      # Inline comment: performs addition  
8     "subtract": lambda x, y: x - y, # Inline comment: performs subtraction  
9     "multiply": lambda x, y: x * y, # Inline comment: performs multiplication  
10 }  
11  
12 # Retrieve the operation function from the dictionary; returns None if not found  
13 func = operations.get(operation) # Inline comment: using dictionary get to avoid repetitive if-elif  
14  
15 # Compute the result if the operation exists; else, set result to None  
16 result = func(a, b) if func else None # Inline comment: compute result dynamically  
17  
18 print(result) # Inline comment: prints 15 as expected
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] [X] | [] [X] [X]

PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/Assignment-13.py"
15
PS D:\AI Assisted Coding>

Explanation:

The code begins by defining three variables:

- **operation** is set to "multiply", indicating the mathematical operation that will be performed.
- **a** and **b** are set to 5 and 3, respectively, and serve as the operands for the selected operation.

A dictionary named **operations** is then created to map operation names (as keys) to corresponding lambda functions (as values). Each lambda function takes two parameters (`x` and `y`) and performs a mathematical operation:

- "add" maps to a lambda function that returns the sum ($\lambda x y. x + y$).
- "subtract" maps to a lambda function that returns the difference ($\lambda x y. x - y$).
- "multiply" maps to a lambda function that returns the product ($\lambda x y. x * y$).

The code retrieves the function corresponding to the **operation** by using the dictionary's **get** method:

- `func = operations.get(operation)` tries to find the lambda function for "multiply". If the key doesn't exist, it returns `None`.

Next, the code computes the **result**:

- If the retrieved function (`func`) is not `None`, it calls the lambda with the operands `a` and `b`.
- If `func` is `None`, **result** is set to `None`.

In this case, since "multiply" exists in the dictionary, the lambda function calculates $5 * 3$, resulting in 15.

Finally, the code prints the **result**, which outputs **15** to the console.

Task 5:

Prompt:

Refactor the following legacy code to simplify the search logic using Python's in keyword or other efficient techniques. Ensure the output remains the same

```
items = [10, 20, 30, 40, 50]
```

```
found = False
```

```
for i in items:
```

```
    if i == 30:
```

```
        found = True
```

```
        break
```

```
print("Found" if found else "Not Found")
```

Expected Output:

Found

Code & Output:

```
Assignment-13.py > ...
1  # Using Python's 'in' keyword for an efficient search
2  items = [10, 20, 30, 40, 50]
3  # Check if 30 is in the list, which internally optimizes the search
4  found = 30 in items # This replaces the nested loop search
5
6  # Print the result based on whether 30 was found
7  print("Found" if found else "Not Found")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] ... | [] X

PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/Assignment-13.py"

Found

PS D:\AI Assisted Coding>

Explanation:

The code works as follows:

- 1. List Definition**
The variable `items` is a list containing integer values.

```
items = [10, 20, 30, 40, 50]
```

This list is the collection in which the program will search for a specific element.
- 2. Efficient Search Using `in` Keyword**
Instead of using a nested loop, the code utilizes Python's `in` operator to check if the value `30` exists in the list.

```
found = 30 in items # This replaces the nested loop search
```

 - The expression `30 in items` evaluates to `True` if `30` is found in the list, otherwise it returns `False`.
 - This approach is more readable and efficient for linear searches compared to manually iterating over the list with loops.
- 3. Conditional Printing**
The code then prints `"Found"` if `found` is `True` (meaning `30` exists in the list) or `"Not Found"` if `found` is `False`.

```
print("Found" if found else "Not Found")
```